



Multi-Agent & Kết Nối Hệ Thống

AICB-P1 · Ngày 9 · MCP, A2A & LangGraph

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ...

“Bạn có 1 agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent. Làm thế nào để hệ thống vẫn rõ vai trò, dễ kiểm soát, và dễ mở rộng?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

D1
Prompt

D2
RAG

D3
Eval

D4
Tools

D5
Memory

D6
Safety

D7
Prod.

D8
Adv. RAG

D9
Multi-Agent

Vị trí của hôm nay

Day 08 đã dạy cách **lấy đúng thông tin**. Day 09 hỏi câu tiếp theo: khi bài toán lớn hơn một agent, ta **tổ chức hệ thống** như thế nào?

1. Giới hạn của single-agent
2. Mental model: tư duy hệ thống
3. Multi-agent patterns
4. Cơ chế điều phối: ai giữ kế hoạch?
5. Supervisor-worker deep dive
6. MCP – chuẩn kết nối tool
7. A2A – giao tiếp giữa agents
8. Orchestration với LangGraph
9. Observability & debugging
10. Cost, latency & reliability
11. An toàn & chế độ hỏng
12. Kiến trúc thật ngoài đời
13. Harness engineering
14. Tự chủ dài hạn
15. Kế hoạch học tập

Mạch xuyên suốt

một agent → nhiều agent → kết nối ra ngoài (MCP/A2A) → điều phối (LangGraph) → quan sát được (trace) → **chạy được lâu và đọc bằng chứng cho đúng.**

- Giải thích được vì sao **single-agent** bắt đầu quá tải khi bài toán cần nhiều vai trò và nhiều nguồn lực
- Phân biệt được các pattern **supervisor-worker**, **pipeline**, **debate**, và **hierarchical** – và biết chọn đúng pattern
- Hiểu **MCP** là chuẩn nối agent với tool / service bên ngoài, và **A2A** là cách agents giao việc cho nhau với message contract rõ
- Dùng **LangGraph** để hình dung graph, state, và conditional routing trong hệ multi-agent
- Thiết kế được **trace & observability** để debug và cải thiện hệ thống
- Thiết kế được trên giấy một hệ **Supervisor + Workers** có trace rõ ràng, từ một pipeline RAG có sẵn
- Biết **harness** là gì, vì sao nên sửa harness trước khi đổi model, và đọc bằng chứng về **tự chủ dài hạn** một cách phê phán
- Chỉ ra được các **chế độ hỏng** đặc thù của hệ nhiều agent và các **ranh giới quyền hạn** cần đặt trước khi cho agent hành động thật

Từ Single-Agent Sang Multi-Agent

Day 08 giúp agent biết retrieve và trả lời grounded; Day 09 trả lời câu hỏi khi nào một agent không còn đủ để gánh toàn bộ bài toán

Day 08 đã làm được

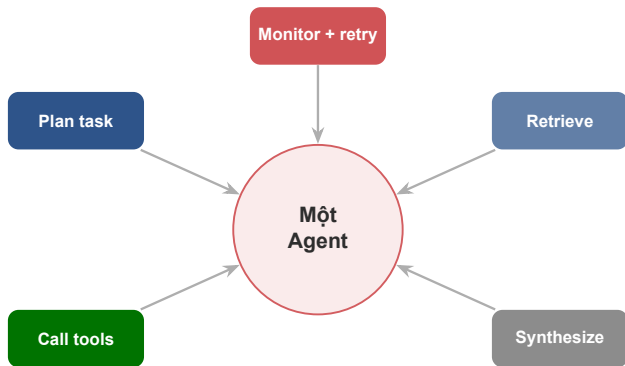
- retrieve đúng tài liệu hơn
- rerank hoặc lọc context tốt hơn
- generate câu trả lời grounded hơn

Nhưng khi hệ thống lớn lên

- phải phân tích task trước khi retrieve
- phải gọi thêm tool ngoài
- phải chia việc và tổng hợp nhiều kết quả
- phải theo dõi trace để debug

Thông điệp mở bài

Day 09 không phủ định Day 08. Nó là bước tiếp theo: **biến một agent có RAG thành một hệ thống có vai trò, có phối hợp, và có điểm mở rộng rõ ràng.**



Một nơi phải làm quá nhiều việc sẽ khó tối ưu, khó debug, và khó scale.

Lưu ý: Câu hỏi đúng không còn là “agent có đủ thông minh không?” mà là “ta có đang ép một agent gánh quá nhiều vai trò không?”

1. Context bottleneck

Một agent phải giữ quá nhiều mục tiêu, tool outputs, evidence, và state trong cùng một lần suy luận. Context window có giới hạn cứng.

2. Specialization trade-off

Agent càng ôm nhiều vai, prompt càng dài và khó ổn định. Giỏi đều mọi thứ thường đồng nghĩa với không thật sự giỏi vai nào.

3. Parallelism hạn chế

Một agent thường chạy tuần tự. Khi có nhiều việc độc lập, hệ thống vẫn phải chờ từng bước nối nhau – tăng latency không cần thiết.

4. Reliability yếu

Nếu agent chọn sai tool hoặc hiểu sai task ở đầu luồng, toàn bộ hệ thống dễ đi lệch theo. Không có isolation để khoanh vùng lỗi.

Kịch bản thực tế

- Giả sử context window **24k tokens**; task: phân tích hợp đồng 80 trang + tra luật
- Tool call trả về 12,000 tokens
- Chat history đã chiếm 6,000 tokens
- Prompt gốc: 3,000 tokens
- Còn lại cho reasoning: $\approx 3,000$ tokens

Dấu hiệu nhận biết

- câu trả lời thiếu thông tin ở giữa document
- agent lặp lại bước đã làm
- reasoning ngắn bất thường ở bước cuối
- tool call với empty context

Lưu ý: Agent bắt đầu “quên” phần đầu tài liệu khi xử lý phần cuối – đây là **lost-in-the-middle problem**.

Nguyên tắc nền (Anthropic, 2024)

- “Chỉ thêm hệ agentic nhiều bước **khi giải pháp đơn giản hơn đã không đủ.**”
- Tối ưu một LLM call với retrieval + ví dụ thường đã giải quyết phần lớn bài toán.
- Agentic tự chủ $\Rightarrow$ chi phí cao hơn và lỗi dễ **cộng dồn.**

Cái giá phải trả

Nhiều agent = nhiều LLM call = nhiều token. Con số cụ thể nằm ở phần **Cost & Reliability** – multi-agent có thể tốn **hơn cả chục lần** token so với một lần chat.

Lưu ý: Multi-agent không phải là “bản nâng cấp miễn phí”. Nó là một **đánh đổi** bạn chọn khi bài toán thực sự cần – không phải vì nó nghe “ngầu”.

- ✓ **Task có nhiều bước vai trò khác nhau:** plan, retrieve, tool use, tổng hợp
- ✓ **Có thể chia việc độc lập:** 2 worker làm song song vẫn hợp lý
- ✓ **Cần debug rõ ai làm sai:** route sai, retrieve sai, hay synthesis sai
- ✓ **Cần mở rộng dần:** thêm 1 worker mới mà không viết lại cả prompt gốc
- ✓ **Context window một agent không đủ:** tài liệu lớn, nhiều tool output
- ☐ **Đừng dùng multi-agent chỉ vì thấy “ngầu”.** Nếu 1 workflow đơn giản đã đủ, giữ đơn giản sẽ rẻ và ổn định hơn

Mental Model: Tư Duy Hệ Thống

Trước khi chọn pattern hay tool, cần hình thành tư duy đúng về cách chia trách nhiệm trong một hệ thống phức tạp

Tư duy cũ

- Làm thế nào để agent thông minh hơn?
- Prompt nào khiến agent làm được nhiều hơn?
- Thêm nhiều tool cho một agent để đủ sức?

Tư duy hệ thống

- Task này gồm bao nhiêu loại trách nhiệm khác nhau?
- Ai cần biết gì, khi nào?
- Lỗi cần được khoanh vùng ở đâu?
- Điểm nào cần human oversight?

Lưu ý: Tư duy cũ dẫn đến **“god agent”** – một agent làm hết nhưng không ai hiểu nó đang làm gì. Tư duy hệ thống cho ta vai trò rõ, để test từng phần, để cải thiện dần.

Câu hỏi 1 – Chia trách nhiệm

Task nào cần reasoning?
Task nào cần data fetch-
ing? Task nào chỉ cần for-
mat?

Câu hỏi 2 – Thông tin đi đườn

Agent nào cần biết gì? Ai
cần đầu ra của ai trước
tiên?

Câu hỏi 3 – Lỗi ở đâu ít tổn hại

Thiết kế để lỗi tại một
worker không làm hỏng
toàn bộ hệ thống.

Vì sao quan trọng

Thiết kế tốt giúp lỗi có **địa chỉ rõ ràng** thay vì lan ra cả hệ thống.

Anthropic (2025): multi-agent thắng

Hệ research nhiều agent (lead + 3–5 subagent song song) mạnh ở việc **đọc rộng**: nhiều hướng độc lập, vượt quá một context window.

Cognition (2025): cẩn thận

Subagent song song dễ **ra quyết định ngầm mâu thuẫn** khi cùng **ghi** vào một sản phẩm chung (ví dụ: 1 subagent vẽ nền kiểu Mario, subagent kia làm con chim Flappy Bird) – nên ưu tiên một luồng liên tục.

Khung quyết định (tổng hợp 2025)

Song song hoá việc đọc, tuần tự hoá việc ghi. Nếu các agent phải thống nhất trên một artifact chung → ưu tiên một luồng; nếu chúng khám phá các hướng độc lập → chạy song song.

Bài báo	Ý tưởng cốt lõi	Còn dùng 2026?
ReAct (2022)	xen kẽ suy nghĩ và hành động trong cùng một luồng token	Có – là vòng lặp agent hôm nay
Self-Refine (2023)	tự sinh phản hồi cho chính mình rồi sửa lại	Có, nhưng có điều kiện (slide sau)
Reflexion (2023)	tự phê bình bằng lời, lưu vào bộ nhớ episodic làm “tín hiệu học”	Ý tưởng còn, số liệu gốc không vững
Tree of Thoughts (2023)	tìm kiếm trên cây các hướng suy luận, có quay lui	Đã ngắm vào model reasoning
Toolformer (2023)	model tự học <i>khi</i> nào gọi API	Đã bị thay bởi tool schema khai báo (MCP)
Voyager (2023)	thư viện kỹ năng tự lớn dần, tái sử dụng được	Ý tưởng còn (procedural memory)
CoALA (2023)	khung nhận thức: phân loại bộ nhớ + không gian hành động	Có – là từ vựng chuẩn về bộ nhớ

Bảy ý tưởng nền – gần như mọi framework hôm nay đều là tổ hợp của chúng

Working

Context window hiện tại – vở nháp.

Episodic

Các lần chạy trước: lịch sử hội thoại, trace.

Semantic

Kiến thức về thế giới – **chính là RAG**.

Procedural

Kỹ năng: tool, code, quy trình đã học.

Bạn đã có hai loại rồi

Day 08 cho bạn **semantic** (vector store) và **episodic** (lịch sử hội thoại). Loại bạn **chưa từng xây** là **procedural** – agent tự tích lũy kỹ năng dùng lại được. Đó là biên giới thật của phần bộ nhớ hôm nay.

Plan-then-execute

Chia nhỏ bài toán **trước**, rồi chạy theo kế hoạch.

Thắng khi bài toán **nhìn thấy hết** từ đầu: toán, suy luận đóng.

ReAct – xen kẽ

Nghĩ một bước, làm một bước, nhìn kết quả rồi nghĩ tiếp.

Thắng khi môi trường **chỉ lộ thông tin qua hành động**: web, tool, nhiều lượt.

Quy tắc chọn

Câu hỏi không phải “cái nào tốt hơn” mà là: **bạn có biết đủ để lập kế hoạch chưa?**
Kế hoạch lập trên thông tin thiếu sẽ phải vứt đi và lập lại – lúc đó xen kẽ rẻ hơn.

Self-Refine (2023)

Cùng một model: sinh $\rightarrow$ tự nhận xét $\rightarrow$ sửa. Báo cáo cải thiện $\sim 20\%$.

Phản biện (Google, 2023)

Trên bài toán suy luận, model **không tự sửa được** – đôi khi còn **tệ đi** – nếu **không có phản hồi từ bên ngoài**.

Cách hoà giải

Tự phê bình chỉ hiệu quả khi kênh phê bình **chạm được sự thật bên ngoài**: chạy test, tra cứu, hoặc **một model khác với context khác**. $\Rightarrow$ verifier nên là **một vai riêng**. Lưu ý: LLM chấm bài cũng lệch theo **vị trí, độ dài và tự ưu ái** model cùng dòng.

Giống thật

- **Blackboard** (1970s): nhiều chuyên gia cùng đọc/ghi một **bảng chung** → chính là shared state hôm nay
- **Contract Net** (1980): quản lý **đấu thầu** việc cho worker → tổ tiên của supervisor routing

Giống bề ngoài thôi

- **BDI** (1991): niềm tin / mong muốn / ý định – từ vựng hay, nhưng agent LLM **không có** ngữ nghĩa cam kết hình thức
- **Stigmergy**: phối hợp gián tiếp qua dấu vết môi trường – đúng với shared file, **ngược** với A2A nhắn tin trực tiếp

Lưu ý: Vì sao nên biết: phần lớn “phát hiện mới” về điều phối agent đã có tên gọi từ 40 năm trước. Nhưng **đừng ép mọi analogy** – Brooks (1975) giải thích tốt vì sao thêm agent làm chậm hệ; còn nói sang CAP/consensus thì **trực giác nghe hợp lý nhưng chưa ai chứng minh** cho agent LLM.

Multi-Agent Patterns

Có nhiều cách chia hệ thống thành nhiều agent; điều quan trọng là chọn pattern giúp giải quyết đúng loại phức tạp, không tạo thêm phức tạp giả

03

Supervisor-Worker

1 supervisor điều phối nhiều worker chuyên biệt.

Mạnh ở: routing rõ, dễ kiểm soát, dễ trace

Pipeline

Agent A xong rồi mới chuyển output cho B.

Mạnh ở: flow ổn định, tuyến tính, dễ test

Debate

Nhiều agent cùng giải một bài toán rồi vote / synthesize.

Mạnh ở: phản biện, giảm blind spot

Hierarchical

Supervisor lồng supervisor cho nhiều tầng.

Mạnh ở: mở rộng tốt ở enterprise scale

Lựa chọn cho Day 09

Chúng ta đi sâu vào **supervisor-worker** vì đây là pattern dễ hiểu nhất, dễ suy luận về luồng chạy, và là điểm xuất phát hợp lý nhất từ một pipeline RAG.

Pattern	Ý tưởng	Multi-actor?
Prompt chaining	chia task thành chuỗi bước, output bước này là input bước sau	1 LLM
Routing	phân loại input rồi đưa tới nhánh xử lý chuyên biệt	1 LLM
Parallelization	sectioning (chia việc độc lập) hoặc voting (chạy nhiều lần)	có
Orchestrator-workers	1 LLM trung tâm chia việc, giao worker, rồi tổng hợp – = supervisor-worker	có
Evaluator-optimizer	một LLM tạo, một LLM đánh giá & phản hồi trong vòng lặp	1 LLM

Nguồn: Anthropic, “Building Effective Agents” (2024) – framing được cộng đồng dùng rộng rãi

Lý do sự phạm

- học viên dễ nhìn ra vai trò
- dễ nối với use case thật
- dễ giải thích logic route
- nổi thẳng được từ một pipeline RAG có sẵn

Lý do triển khai

- bắt đầu từ 2–3 worker là đủ
- dễ cắm thêm MCP tool worker
- trace và testing rõ hơn
- supervisor thường chỉ cần một LLM call nhỏ

Lưu ý: Nếu Day 09 ôm nhiều pattern ngang nhau, người học sẽ nhớ tên pattern nhưng không biết ngày mai nên build theo pattern nào.

Cơ Chế Điều Phối: Ai Giữ Kế Hoạch?

Phần trước nói về hình dạng của hệ; phần này nói về cơ chế bên dưới – ai quyết bước tiếp theo, và vì sao điều đó đổi cách bạn debug lẫn cách bạn tính tiền



	LLM quyết từng lượt	Peers + task list chung	Script quyết
Ai giữ kế hoạch	agent chính	lead + danh sách việc	đoạn code bạn viết
State trung gian nằm ở	context window	shared task list	biến trong script
Cái lặp lại được	định nghĩa worker	định nghĩa team	chính luồng điều phối
Hợp khi	việc chưa rõ hình dạng	worker cần bàn với nhau	việc lặp lại, viết ra trước được
Chi phí	thấp nhất	cao (mỗi peer một phiên)	theo số agent, nhưng không phình context

Trực “ai giữ kế hoạch” là cách nhanh nhất để phân loại mọi framework bạn sẽ gặp

Agents-as-tools – gọi rồi trả về

Kết quả **trả lại vòng lặp của agent gọi**. Agent gọi **vẫn giữ** cuộc hội thoại và tự tổng hợp tiếp.

Worker chỉ thấy đúng phần input được truyền vào.

Handoff – chuyển quyền lái

Agent kia **trở thành người lái** phần còn lại. Agent cũ **không tự động** lấy lại quyền.

Mặc định agent mới thấy **toàn bộ lịch sử** hội thoại trước đó.

Câu hỏi kiểm tra

Với model, **cả hai đều chỉ là một tool call**. Khác nhau ở chỗ runtime làm gì *sau* đó – hãy hỏi: **“Sau lời gọi này, tới lượt ai nói với user?”** Vẫn là agent gọi → agents-as-tools; là agent được gọi → handoff. (*Lựa chọn thứ ba: cố định luồng trong code – worst-case tốt hơn, kém linh hoạt hơn.*)

Cơ chế	Ai giữ kế hoạch	Lồng nhau	Trần cứng
Subagents	agent chính, từng lượt	sâu tối đa 3 cấp	trả về bản tóm tắt , không phải cả transcript
Agent teams	lead, từng lượt	không lồng được	khuyến nghị 3–5 peer; chi phí tăng tuyến tính
Workflow (script)	script	–	16 agent chạy song song / 1.000 agent mỗi lần chạy
Coordinator (API)	coordinator phía server	đúng 1 cấp	roster ≤ 20 agent, ≤ 25 luồng đồng thời

roster = danh sách agent mà coordinator được phép gọi. Giới hạn theo tài liệu chính thức Anthropic (8/2026) – loại số sẽ đổi, hãy tra lại mỗi kỳ

Cái giá của phối hợp

- lỗi **nhân lên** qua mỗi cấp; phối hợp tập trung (có chốt kiểm) chặn tốt hơn nhiều agent độc lập
- lợi ích của việc thêm agent **bảo hoà** khá sớm, sau đó overhead ăn hết phần lợi
- khảo sát 7 framework (preprint MAST 2025): tỉ lệ hỏng **41–87%**

Vì sao cô lập context lại hiệu quả

Đo trên 18 model lớn (báo cáo Chroma 2025): độ chính xác **tụt trước khi** context đầy – hiện tượng gọi là **context rot**.

Cho mỗi worker một context sạch → tránh được phần tụt đó.

Lưu ý: Đang tranh luận, đừng dạy như chân lý: nhiều người nói subagent mạnh vì mỗi con là “chuyên gia”. Bằng chứng hiện tại nghiêng về lời giải thích khác – cái lợi chính đến từ **context sạch**, không phải chuyên môn hoá. Chưa có thí nghiệm tách bạch hai yếu tố này.

Framework từ chối đoán

Hai nhánh song song ghi cùng một key trong *một bước* → LangGraph **ném lỗi**, không ghi đè âm thầm:

Can receive only one value per step.

Muốn gộp thì phải khai báo

Khai một **reducer** cho key đó (ví dụ “nối thêm vào list”) thì mới có merge. **Không có CRDT** nào ở dưới – bạn tự viết hàm gộp.

Lưu ý: Bẫy tinh vi: chỉ ghi **đồng thời** mới báo lỗi. Ghi **tuần tự** vào key không có reducer thì vẫn **ghi đè im lặng** – worker sau xóa kết quả worker trước mà không ai biết.

Khi không ai biết

- 12,4% lỗi: hệ **không biết điều kiện dừng**
- biểu hiện: **lặp lại bước** (15,7%) hoặc **dừng quá sớm** (6,2%)

Trần độ quy không phải điều kiện dừng

Framework có trần cứng (mặc định **1.000 bước**) – nhưng đó là **túi húng khi rơi**, không phải logic dừng. Graph tốt **không bao giờ** chạm tới nó.

Thiết kế đúng

Phải có **ai đó cầm quyền tuyên bố xong** – một vai có thẩm quyền cuối, hoặc một điều kiện kiểm được bằng máy. Trong một nghiên cứu, chỉ việc **trao quyền quyết định cuối** cho đúng một vai đã tăng tỉ lệ thành công **+9,4%**.

Mặc định nguy hiểm

Ví dụ mẫu **trong** chính tài liệu **Lang-Graph** gộp kết quả bằng cách **nối chuỗi**:

```
"\n---\n".join(sections)
```

Không tóm tắt, không khử trùng lặp → context của bước tổng hợp **phình tuyến tính** theo số worker.

Cách làm đúng (hệ production)

Nén ở worker: mỗi worker là một *bộ lọc thông minh*, trả về **kết luận** chứ không trả cả transcript.

Tổng hợp ở supervisor: một lượt LLM thật sự, không phải phép nối.

Lưu ý: Đây là lý do slide trước nói subagent **trả về bản tóm tắt**. Nối chuỗi là đường dễ đi nhất – và là đường đưa bạn thẳng vào context rot.

Thang chia vai (Anthropic)

- tra cứu đơn giản: **1** agent, 3–10 lượt gọi tool
- so sánh trực tiếp: **2–4** subagent
- nghiên cứu phức tạp: **10+** subagent, vai chia rõ

Hồng ở cả hai đầu

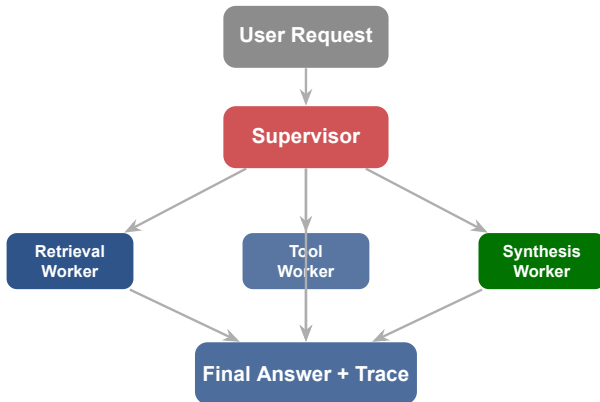
- chia **quá thô** + đề bài mơ hồ → hai worker **làm trùng** nhau
- chia **quá mịn** → đã có bug sinh **50 subagent** cho một câu hỏi đơn giản

Khi worker chết

Framework cho bạn **retry có backoff** và phân biệt được **timeout / hết lượt thử / bị huỷ**. Nhưng **chính sách** thì vẫn là quyết định thiết kế của bạn: thử lại, lập kế hoạch lại, chạy giảm chất lượng, hay dừng hẳn. Lỗi worker trả về None trông **y hệt thành công** – hãy dùng envelope có cấu trúc.

Supervisor-Worker: Deep Dive

Thay vì ép một agent làm hết, ta cho một supervisor phân việc và nhiều worker làm phần việc hẹp, dễ kiểm soát hơn



Khung nghĩ đúng

Supervisor không cần “thông minh hơn tất cả”. Vai trò chính là **chia việc đúng, gọi đúng worker, và gom đầu ra thành kết quả mạch lạc.**

Supervisor

- phân tích yêu cầu ban đầu
- quyết định worker nào nên tham gia
- theo dõi trạng thái và retry nếu cần
- tổng hợp đầu ra cuối cùng
- biết khi nào cần human review

Worker

- xử lý một năng lực hẹp
- nhận input rõ ràng, trả output rõ ràng
- càng stateless càng dễ test
- thất bại cục bộ không làm hỏng cả kiến trúc
- có thể thay thế mà không ảnh hưởng supervisor

Rule of thumb

Supervisor giữ **decision flow**; worker giữ **domain skill**. Đừng để một worker vừa làm việc hẹp vừa bí mật điều phối cả hệ thống.

Specialized

Một worker nên có một năng lực chính: retrieve, gọi tool, tóm tắt, kiểm tra policy...

Stateless ưu tiên

Nếu có thể, worker chỉ cần input hiện tại thay vì ôm cả lịch sử hệ thống.

Testable

Có input / output rõ để test độc lập trước khi cắm vào supervisor.

Lưu ý: Worker mơ hồ thường làm debugging cực khó: không biết lỗi do route sai, prompt sai, hay contract đầu vào chưa đủ rõ.

God Supervisor

Supervisor làm quá nhiều: plan, retrieve, synthesize, monitor. Nó trở thành single-agent được đổi tên.

Implicit State

State bị truyền qua biến toàn cục hoặc side effect. Không ai biết hệ đang ở bước nào.

Chatty Workers

Workers liên tục gọi ngược lại supervisor để hỏi thêm. Message overhead tăng nhanh.

No Fallback

Worker gặp lỗi và không trả về gì. Supervisor chờ mãi không thấy đầu ra để tổng hợp.

Shared state

- dễ xem toàn cảnh
- tiện cho graph orchestration (LangGraph)
- nhưng dễ bị “đụng tay” nếu không có kỷ luật
- cần schema rõ: ai đọc gì, ai ghi gì

Message passing

- contract rõ hơn giữa các agents
- ít coupling hơn
- nhưng phải thiết kế message format cẩn thận
- cần validation ở mỗi điểm nhận

Cách dạy thực dụng

Trong Day 09: **shared state giúp điều phối**, còn **message contract giúp giao tiếp không nhập nhằng**. Day 09 dùng shared state (hợp với LangGraph) làm trục chính.

Những trường cần có trong shared state

- task: task gốc từ user
- plan: danh sách worker cần gọi
- worker_results: output từng worker
- status: pending | running | done | error
- final_answer: tổng hợp cuối
- trace: log dạng list có timestamp

Vì sao trace là bắt buộc?

Không có trace trong state, sau khi hệ chạy xong không ai biết agent đã đi con đường nào để ra kết quả đó.

Một mức chi tiết hơn

Đây là schema **tối thiểu, mang tính khái niệm**. Khi làm thật, worker_results thường được **tách thành từng trường riêng** cho mỗi worker (retrieval_result, tool_result, synthesis_draft) để dễ trace và dễ test từng worker độc lập. Cùng ý tưởng, khác độ mịn.

MCP – Model Context Protocol

Nếu supervisor-worker là cách chia người làm việc, thì MCP là cách agent cắm được vào năng lực bên ngoài mà không phải custom từng tích hợp từ đầu

Trước MCP – vấn đề thực tế

- mỗi tool cần một adapter riêng
- đổi API của tool = viết lại code tích hợp
- mỗi framework gọi tool một kiểu khác nhau
- không có chuẩn chung để agent biết tool làm gì

MCP giải quyết

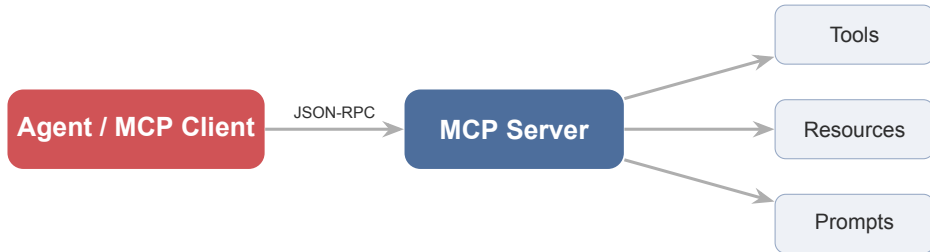
Một chuẩn giao tiếp duy nhất giữa agent và tool. Agent biết cách “khám phá” capability mà không cần hard-code từng tích hợp.

Lưu ý: Điều quan trọng với người học là hiểu **vì sao MCP giúp mở rộng hệ thống**, không phải học thuộc protocol spec trong buổi đầu tiên.

- **MCP** là một chuẩn (Anthropic khởi xướng, mở mã 11/2024; **đã hiến tặng cho Agentic AI Foundation / Linux Foundation 12/2025**) để agent kết nối với external capabilities.
- Thay vì mỗi tool một kiểu tích hợp, agent nói chuyện với một **MCP server** qua **JSON-RPC**.
- MCP server công bố **tools**, **resources**, và **prompts**.
- Agent có thể list, describe, và invoke các capability đó theo chuẩn chung.

Analogy dễ nhớ

Supervisor-worker nói về **vai trò**.
MCP nói về **ổ cắm chuẩn** để agent dùng tài nguyên bên ngoài – giống **USB**: mọi thiết bị cùng một chuẩn kết nối.



Điểm cốt lõi

Mỗi host chạy nhiều client, mỗi client nối **1:1** tới một server. Agent không cần biết chi tiết hệ thống phía sau – nó chỉ cần hiểu **MCP surface** mà server công bố.

Tools (POST-like)

Hành động / thao tác.
Ví dụ: search, query API,
tạo ticket, gọi webhook

Resources (GET-like)

Tài nguyên để đọc.
Ví dụ: file, schema, cata-
log, config, DB

Prompts

Prompt dùng lại.
Giúp chuẩn hóa cách gọi
năng lực và giảm lỗi
prompt

Lưu ý: Không phải MCP server nào cũng có đủ cả ba. Điều quan trọng là agent có một **cách nhìn nhất quán** về các capability được công bố.

stdio

JSON-RPC qua stdin/stdout cho server chạy **local** (subprocess). Client nên hỗ trợ khi có thể.

Streamable HTTP

Một endpoint (POST + GET), tùy chọn streaming. Cho server **remote**; phải validate Origin để chống DNS-rebinding.

Lưu ý: Lưu ý 2026: Streamable HTTP (3/2025) đã **thay thế** transport HTTP+SSE cũ – SSE chỉ là fallback deprecated. Bản spec 7/2026 còn bỏ luôn session ở tầng giao thức: **MCP đi theo hướng stateless**, cùng lý do ta thiết kế worker stateless. *Chi tiết spec đổi nhanh – tra changelog trước khi build.*

Luồng discovery

1. Agent kết nối tới MCP server
2. Gọi `tools/list` để lấy danh sách tool
3. Mỗi tool trả về: `name`, `description`, `inputSchema`
4. Agent đọc schema, chọn tool phù hợp
5. Gọi `tools/call` với đúng parameters
6. Server thực thi và trả kết quả

Tại sao quan trọng?

Agent không cần được lập trình sẵn biết tool “X” tồn tại. Nó có thể **khám phá khi chạy** và tự điều chỉnh theo tool nào có sẵn.

Kịch bản: Customer Support Agent

- Tool Worker cần tra policy mới nhất
- Gọi MCP server “knowledge-base”
- Server expose tool `search_policy`, `get_faq`
- Worker gọi `search_policy(query, date_after)`
- Kết quả JSON chuẩn kèm source

Lợi ích trực tiếp

- team đổi policy → chỉ update MCP server
- supervisor/worker không cần sửa khi tool đổi
- thêm tool mới = thêm endpoint vào server
- dễ audit ai gọi tool gì, khi nào

Hội tụ cross-vendor (2025)

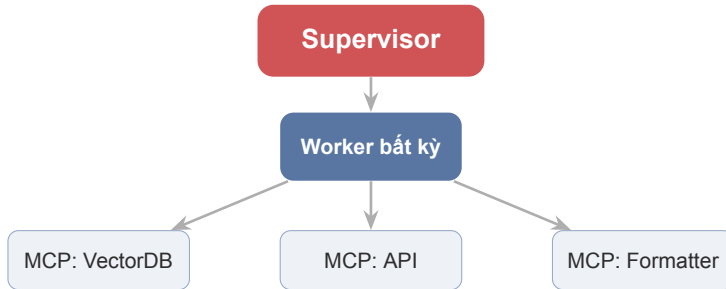
- **OpenAI**: MCP trong Agents SDK + Responses API
- **Google**: hỗ trợ Gemini, công bố tại I/O (5/2025)
- **Microsoft**: GA trong Copilot Studio

Hội tụ là thật; nhưng độ sâu hỗ trợ mỗi bên khác nhau.

Security cần biết

- OAuth 2.1 (draft) + token audience-bound
- **tool poisoning**: mô tả tool độc hại có thể lái agent
- **prompt injection** từ tool output

Mitigation là client-side: coi output là untrusted, duyệt tool rủi ro, sandbox.



Điểm chốt section

Sơ đồ supervisor-worker vẫn như cũ – chỗ mới là **lớp MCP nằm giữa** worker và capability thật. Worker chỉ cần biết MCP surface, không cần biết hệ phía sau là gì: đó là **ecosystem effect**.

A2A – Agent to Agent Communication

MCP giúp agent nói chuyện với tool; A2A giúp agent nói chuyện với agent khác theo cách rõ nhiệm vụ, rõ bối cảnh, và rõ đầu ra mong đợi

MCP – agent ↔ tool

- agent nói chuyện với **tool / capability**
- mục tiêu: kết nối năng lực bên ngoài
- tool không có agency – chỉ thực thi

A2A – agent ↔ agent

- agent nói chuyện với **agent khác**
- mục tiêu: chia việc và đồng bộ
- agent phía kia **có thể ra quyết định**

Cách nhớ (chính thống: “A2A loves MCP”)

MCP trả lời “agent lấy năng lực ở đâu?” (chiều dọc, ra ngoài). **A2A** trả lời “agent giao việc cho ai?” (chiều ngang, giữa các agent). Hai thứ **bổ sung**, không cạnh tranh.



- Donate cho **Linux Foundation** (vendor-neutral); founding members: AWS, Cisco, Google, Microsoft, Salesforce, SAP, ServiceNow
- IBM **ACP** (Agent Communication Protocol) đã gộp vào A2A (8/2025) – hội tụ về một chuẩn agent-to-agent
- **MCP cũng đã về Agentic AI Foundation** – một quỹ trực thuộc Linux Foundation (12/2025)
⇒ governance **không còn** là điểm khác biệt giữa hai chuẩn; điểm khác biệt là **tầng** mà mỗi chuẩn giải quyết

Lưu ý: Phân biệt 2 mốc: A2A được **công bố** 4/2025, nhưng **donate** cho Linux Foundation 6/2025. Con số adoption (100+ công ty, 22K+ sao) là ảnh chụp **4/2026** từ thông cáo – hãy ghi kèm mốc thời gian, đừng đọc như số hiện tại.

Agent Card

Một JSON metadata agent công bố: identity, capabilities, **skills**, endpoint, auth. Client agent đọc card để biết **ai làm được task này**.

Đường dẫn quy ước:
`/.well-known/agent-card.json`

Core objects

- Task: đơn vị công việc có lifecycle (submitted → working → done/failed)
- Message: một lượt trao đổi (user/agent)
- Part: text / file / structured data
- Artifact: kết quả tạo ra

Nguyên tắc thiết kế

Ba binding chính thức: **JSON-RPC 2.0**, **gRPC**, **HTTP+JSON/REST** (SSE cho streaming). Agent là **peer hộp đen** (opaque): cộng tác mà **không chia sẻ memory / tool** nội bộ của nhau.

Task

Agent kia cần làm gì?
Ví dụ: tìm 3 chunk policy
phù hợp nhất

Context

Worker cần biết gì để
làm đúng?
query, constraints, user
role, state

Expected output

Trả về theo format nào?
list chunks, score, ratio-
nale, error

Ví dụ:

task = "retrieve evidence"

context = "user hỏi về refund policy, ưu tiên tài liệu sau 2025"

expected_output = "top 3 chunks + source + confidence"

"Need to know": thiếu → worker trả sai; thừa → tốn token, đổi schema là vỡ. Không chắc thì bắt đầu ít hơn.

Sync

- đơn giản để hiểu và debug
- hợp khi supervisor cần kết quả ngay
- nhưng dễ tăng latency toàn luồng
- **bắt đầu ở đây cho Day 09**

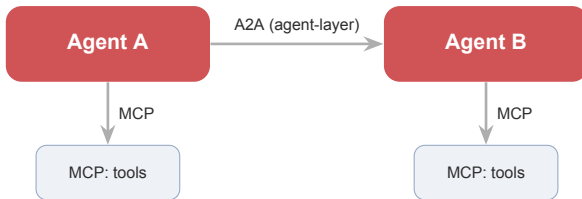
Async

- hợp khi nhiều worker chạy song song
- tận dụng được concurrency
- nhưng cần quản lý trạng thái và timeout tốt hơn
- mở rộng khi đã nắm sync tốt

Cách giảng đơn giản

Sync dễ dạy cho vòng đầu. Async chỉ cần giới thiệu như một hướng mở rộng khi nhiều worker có thể chạy độc lập.

- ☒ **Biết rõ ai được gọi ai:** không phải agent nào cũng được chạm mọi capability
- ☒ **Biết dữ liệu nào được truyền đi:** tránh đẩy thừa PII hoặc state nhạy cảm
- ☒ **Biết output nào cần xác thực lại:** đặc biệt khi worker chạm tool ngoài
- ☒ **Validate đầu ra trước khi dùng:** worker có thể trả về schema sai
- ☐ **Đừng giả định mọi agent đều đáng tin như nhau.** Hệ nhiều agent vẫn cần trust boundary



Hai lớp, hai vấn đề

Một app dùng **A2A** để nói chuyện với agent khác; mỗi agent dùng **MCP** để gọi tool của riêng mình. ACP (IBM) đã gộp vào A2A – hệ sinh thái đang hội tụ về 2 lớp này.

Lưu ý: Phân biệt: hệ supervisor-worker nội bộ thường chỉ mượn ý tưởng message contract của A2A – không phải cài đặt A2A thật.

Orchestration Với LangGraph

Sau khi hiểu vai trò và giao tiếp, ta cần một cách biểu diễn luồng chạy rõ ràng; LangGraph là cách rất trực quan để làm điều đó

Không có framework

- routing logic nằm trong một prompt điều phối dài
- khó biết hệ đang ở bước nào
- thêm một nhánh mới = sửa cả prompt
- debug bằng `print()` và hy vọng

Với LangGraph

Routing trở thành **code tường minh**. Graph visualize được, state có schema, human-in-the-loop có điểm rõ ràng.

Bối cảnh 2026

LangGraph **1.0 GA** (10/2025). Các primitive lõi (StateGraph, nodes, edges) ổn định; khi build nên pin version để code chạy đúng.

Node

- là một hàm Python nhận state, trả về phần state cần cập nhật
- tương ứng một agent / một bước (supervisor, worker, human review)

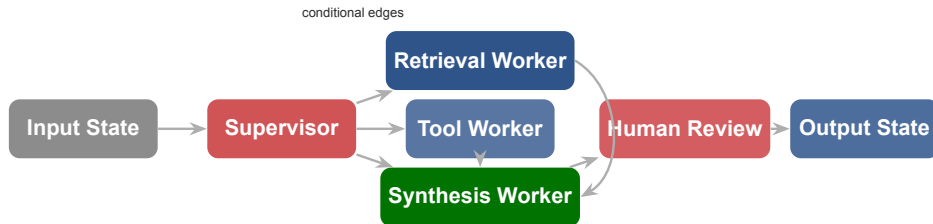
Edge

- **unconditional**: luôn đi $A \rightarrow B$
- **conditional**: hàm trả tên node tiếp theo dựa trên state

State

TypedDict (dict Python có schema kiểu tĩnh; hoặc Pydantic) truyền qua mỗi node. Mỗi node đọc toàn bộ state, ghi vào các trường được phép.

node = ai làm · edge = đi đâu · state = hệ biết gì



Điểm cần nhớ

LangGraph làm rõ **route** quyết định ở đâu, **state** đi qua đâu, và **human** can thiệp ở điểm nào.

```
from langgraph.graph import StateGraph, START
from typing import TypedDict

class AgentState(TypedDict):
    task: str
    need_tool: bool
    need_retrieval: bool
    trace: list

def route(state) -> str:      # supervisor's decision
    if state["need_tool"]:    return "tool_worker"
    if state["need_retrieval"]: return "retrieval_worker"
    return "synthesis_worker"

g = StateGraph(AgentState)
g.add_node("supervisor", supervisor_node)  # + cac worker node
g.add_edge(START, "supervisor")
g.add_conditional_edges("supervisor", route)
app = g.compile()
```

Routing trở thành logic tường minh, thay vì ẩn trong một prompt điều phối dài. (Các worker node được add tương tự.)

```
def supervisor_node(state: AgentState) -> AgentState:
    decision = llm.invoke(
        SUPERVISOR_PROMPT.format(task=state["task"])
    )
    return {
        **state,
        "need_retrieval": decision.need_retrieval,
        "need_tool": decision.need_tool,
        "trace": state["trace"] + [
            f"[supervisor] tool={decision.need_tool}"
        ],
    }
```

Supervisor node **không làm việc của worker** – chỉ ra quyết định route và **ghi trace**. Mọi trường nó dùng đều có trong AgentState.

```
from langgraph.checkpoint.memory import InMemorySaver

# compile WITH a checkpointer -> enables memory + resume
app = g.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "session-1"}}
app.invoke(state, config)  # same thread_id => same conversation
```

- Checkpointer cho phép **multi-turn memory** và **resume sau interrupt** – cùng thread_id = cùng cuộc.
- InMemorySaver khi thử; production dùng SqliteSaver / PostgresSaver (cài gói riêng).

```
from langgraph.types import interrupt, Command

def human_review(state: AgentState):
    # pause the graph; a human inspects, then we resume
    interrupt({"last_step": state["trace"][-1]})
    return {"trace": state["trace"] + ["[human] approved"]}

# resume the SAME thread (app compiled with checkpointers above):
config = {"configurable": {"thread_id": "session-1"}}
app.invoke(Command(resume="approve"), config)
```

Idiom 2026: interrupt() động trong node + Command(resume=) – **cần checkpointer** (frame trước). interrupt_before tính chỉ còn để debug. Node **chạy lại từ đầu** khi resume – giữ side-effect idempotent.

Subagents

Worker có **context riêng**, chỉ **report kết quả** về agent chính – không nói chuyện với nhau.

≈ **supervisor-worker**

Agent Teams

Nhiều phiên Claude **ngang hàng**, chia **task list chung**, **nhắn trực tiếp** cho nhau.

≈ **A2A** (agent ↔ agent)

Dynamic Workflows

Một **script** giữ kế hoạch (loop, nhánh, kết quả) điều phối **hàng chục-trăm agent**.

≈ **orchestration** (như Lang-Graph)

Vẫn là trục “ai giữ kế hoạch?”

Đúng ba nhánh đã học ở phần Cơ chế điều phối, nay mang tên sản phẩm thật: **Subagents** và **Teams** – Claude/lead quyết từng lượt; **Workflow** – script quyết.

*Nguồn: code.claude.com/docs (8/2026): subagents & workflows = GA; agent teams = experimental; Managed Agents = beta. Ba chi tiết đáng nhớ: subagent trả bản tóm tắt chứ không phải transcript · subagent **không nói chuyện với nhau** · replay workflow dừng ở agent dở dang đầu tiên.*

- ☑ **Sản phẩm thật quét prompt-injection trên báo cáo của subagent** trước khi agent cha đọc – vì worker có thể đã đọc file hoặc trang web chứa chỉ thị lạ
- ☑ **Một agent không được duyệt quyền thay agent khác:** lời “đồng ý” chuyển tiếp từ agent khác vẫn bị coi là input không tin được
- ☑ **Coi subagent như một kênh dẫn nội dung ngoài vào orchestrator** – giống hết tool result, không phải “người nhà nên tin”
- ☐ **Đừng nghĩ “agent của tôi thì output của nó đáng tin”.** Đây là đúng cái lỗ hổng worker-tấn-công-supervisor mà giới nghiên cứu đã chứng minh

Observability & Debugging Hệ Multi-Agent

Hệ nhiều agent khó debug hơn nhiều so với single agent; observability tốt là điều kiện tiên quyết để cải thiện được hệ thống

Nguồn gốc lỗi khó xác định

- lỗi có thể từ: routing sai, context sai, tool fail, synthesis sai
- lỗi ở bước A có thể chỉ lộ ra ở bước C
- nhiều agent = nhiều LLM call = nhiều điểm fail tiềm năng

3 câu hỏi observability

1. Agent nào đã chạy, theo thứ tự nào?
2. Input/output tại mỗi bước là gì?
3. Lỗi hay warning nào đã xảy ra?

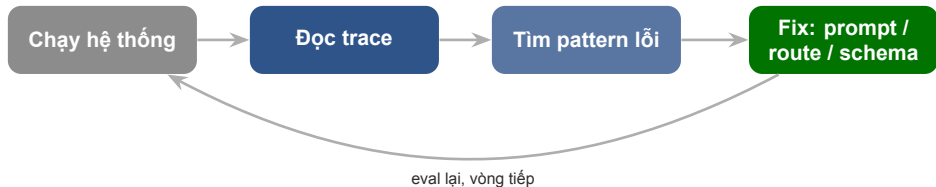
Lưu ý: Không có trace tốt, debugging multi-agent gần như là mò mẫm. Đặc biệt: lỗi thường **ngũ nghĩa** (agent hiểu sai output) chứ không phải crash – trace phải ghi I/O thật, không chỉ latency.

Mỗi entry nên có

- timestamp: khi nào
- agent_id: ai làm
- action: route / call / synth / error
- input_summary / output_summary
- status: ok | warn | error
- latency_ms: mất bao lâu

```
{  
  "t": "14:03:21",  
  "agent": "supervisor",  
  "action": "route",  
  "decision": "retrieval_worker",  
  "status": "ok",  
  "latency_ms": 312  
}
```

Trace phải trả lời được: route theo tiêu chí nào · chỗ nào đã retry/fallback · human đã quyết gì.



Trace là dữ liệu, không chỉ là log

Trace không chỉ để debug lỗi hôm nay. Nó là **dữ liệu để cải thiện** routing, worker quality, và message contract theo thời gian.

LangSmith

Của LangChain, framework-agnostic.

Managed-first (không có bản OSS self-host miễn phí).

Langfuse

MIT ở phần core, self-host được (tính năng Enterprise cần license trả phí). Tốt cho data sovereignty.

Arize Phoenix

Open-source, OTEL-native. Thiên về **local dev / debug**.

OpenLLMetry

Apache 2.0, OTEL extensions (Traceloop). Export sang 25+ backend.

Bắt đầu thế nào cho Day 09

Bắt đầu bằng **JSON log tự viết vào state** – học được nhiều nhất về cách hệ hoạt động. Sau đó “graduate” sang một tool để có search, latency stats, và cost dashboard miễn phí.

Cost, Latency & Reliability

Multi-agent không miễn phí; nhiều agent = nhiều LLM call = nhiều tiền và nhiều điểm fail. Cần tư duy trade-off từ sớm

10

Chi phí tăng từ đâu?

- mỗi agent = ít nhất 1 LLM call
- supervisor thường là một LLM call riêng
- context truyền qua state có thể lớn
- retry = gấp đôi cost tại điểm đó

Nguyên tắc tối ưu

Supervisor **không cần** model lớn nhất. Nếu routing đơn giản, dùng model nhỏ cho supervisor và giữ model mạnh cho worker cần reasoning sâu.

~4×

token của một **agent** so với một lần chat

~15×

token của **multi-agent** so với một lần chat

~80%

phần variance hiệu năng (trên BrowseComp) chỉ riêng **token usage** giải thích

Lưu ý: Phần lớn “multi-agent thắng” thực ra là “**tiêu nhiều compute hơn**”. Chỉ đáng cho task giá trị cao. (*Anthropic tự báo cáo multi-agent thắng single-agent ~90% trên eval nội bộ – không phải benchmark độc lập.*)

Khi so ở cùng ngân sách token

Nhiều nghiên cứu độc lập 2025–2026 ép **hai bên tiêu cùng lượng token** rồi so lại. Kết quả: single-agent **ngang hoặc thắng** ở suy luận nhiều bước.

Phần lớn “lợi thế multi-agent” trong các bài blog là **compute chưa được tính vào**.

Kết luận hội tụ

Hình dạng bài toán quan trọng hơn kiến trúc.

Hợp multi-agent: việc chia song song được, hoặc context vượt quá một cửa sổ.

Hợp single-agent: suy luận tuần tự, phụ thuộc chặt giữa các bước.

Lưu ý: Nói đúng mức: **nhất quán** – context rot, tỉ lệ hỏng cao, debate không thắng self-consistency (chạy 1 agent nhiều lần rồi lấy đa số) ở cùng compute (*phần lớn là preprint 2025–26*). **Đang tranh luận** – multi có thắng single nói chung không. **Lời đồn** – “multi-agent nói chung mạnh hơn”.

Cần thiết kế trước

- worker có **timeout** rõ ràng
- supervisor có **retry logic**: thử lại bao nhiêu lần?
- nếu retry cũng fail: **fallback** là gì?
- **partial failure**: tổng hợp với những worker đã thành công

Graceful degradation

Hệ không cần làm tốt mọi trường hợp. Nó cần **thất bại có kiểm soát** và báo cáo rõ ràng khi không đủ tự tin.

Lưu ý: Thất bại cục bộ của một worker **không nên** crash toàn bộ hệ thống – đó là lợi thế chính của việc chia vai trò.

An Toàn & Chế Độ Hỏng

Hệ nhiều agent không chỉ hỏng nhiều hơn – nó hỏng theo những kiểu mà hệ một agent không có; và bề mặt tấn công cũng mở rộng theo mỗi tool ta cắm vào



Nhóm	Tỉ lệ	Kiểu hồng nổi bật
Thiết kế hệ thống	44%	lặp lại bước (16%) · không biết khi nào dừng (12%) · làm sai đề bài (12%)
Lệch pha giữa agent	32%	nói một đằng làm một nẻo (13%) · trôi khỏi nhiệm vụ (7%) · không hỏi lại khi mơ hồ (7%)
Không kiểm chứng	24%	kiểm chứng sai (9%) · kiểm chứng thiếu (8%) · dừng quá sớm (6%)

Nguồn: khảo sát 1.642 trace trên 7 framework mã nguồn mở (MAST, 2025)

Làm rõ vai trò

Viết rõ agent nào chịu trách nhiệm gì →
+9,4% tỉ lệ thành công.

Thêm một bước kiểm chứng

Chèn một vai *verifier* trước khi kết thúc →
+15,6%.

Câu quan trọng nhất của nghiên cứu này

Tỉ lệ hỏng **không** chỉ do model yếu: *cùng một model*, thiết kế hệ tốt hơn cho kết quả tốt hơn. Phần lớn lỗi nằm ở **chỗ ta thiết kế**, không phải chỗ model nghĩ.

Ba chân

- chạm dữ liệu riêng tư
- đọc nội dung không tin được
- có đường gửi ra ngoài

Vì sao chia agent *không* cứu được

Worker A đọc dữ liệu nội bộ, worker B đọc web, supervisor có tool gửi mail.

Không agent nào có đủ ba chân – nhưng **cả hệ** thì có.

Lưu ý: Đúng hình dạng của một sự cố thật (5/2025): một issue **công khai** có chứa chỉ thị ẩn khiến agent đọc **repo riêng tư** rồi đẩy nội dung ra một **pull request công khai**. Quyền hạn phải tính trên **cả đồ thị**, không phải từng agent.

Tool poisoning

Mô tả tool chứa chỉ thị độc mà UI không hiện, nhưng model đọc đủ.

Shadowing

Server độc mô tả **cách một tool khác** phải hành xử → cướp tool đáng tin.

Rug-pull

Tool hiện lúc được duyệt, **đổi mô tả** sau khi đã có lòng tin.

Điểm chung

Cả ba đều tấn công vào **văn bản mô tả tool** – thứ ta thường không review như review code. Mặc định đúng: **pin** phiên bản tool, đọc mô tả như đọc code, và cấp quyền theo từng worker chứ không cấp cho cả hệ.

Cách làm	Ý tưởng	Trạng thái
Sandbox (microVM / syscall)	cô lập ở mức phần cứng hoặc mức syscall	đã dùng production
Quyền tối thiểu theo worker Cổng duyệt của con người	mỗi worker chỉ có đúng tool nó cần chặn trước hành động rủi ro	có sẵn, ít ai làm đã dùng, không đồng đều
Phân cấp chỉ thị Spotlighting	dạy model ưu tiên lệnh hệ thống đánh dấu rõ đâu là dữ liệu, đâu là lệnh	nghiên cứu nghiên cứu
Dual-LLM (CaMeL)	tách luồng điều khiển khỏi luồng dữ liệu	nghiên cứu, chưa có bản production

Đừng dạy phòng thủ nghiên cứu như thể nó đã có sẵn để dùng

Kiến Trúc Thật Ngoài Đời

Bốn hệ thống đã chạy production, mỗi hệ bị một ràng buộc khác nhau ép phải thiết kế theo một kiểu khác nhau – đọc kiến trúc người khác là cách nhanh nhất để thấy trade-off thật

12

Bốn lớp

- kho log **chỉ ghi thêm**, hàng TB mỗi ngày
- **MCP server** mở kho đó cho agent
- system prompt dựng quanh **đường tấn công** cụ thể
- đội agent sẵn lỗi, có **trí nhớ tổ chức**

Ràng buộc quyết định thiết kế

Độ trễ truy vấn quyết định độ sâu điều tra: 200ms hay 2s là khác biệt giữa thử **3** giả thuyết và **30**. Nên họ đầu tư lớp dữ liệu *trước*, agent sau.

Kết quả công bố & cách duyệt

Độ phủ phát hiện **75** → **400+**; cảnh báo mỗi tuần **~300k** → **~20k**. Mọi thay đổi vào production đều đi qua **pull request có người duyệt** – không bao giờ tự động áp.

Thiết kế

Một cổng MCP hợp nhất **45+ hệ thống** sau một giao diện; kiểm soát truy cập **nhiều tầng**; log kiểm toán đầy đủ.

Chỗ nào agent đã biết dùng CLI thì **expose CLI**, không dựng MCP server mới.

Bài học xương máu

- “phủ 80% quy trình **mang lại giá trị bằng không**”
- agent **rất thích ăn gian** → phải có vòng người kiểm
- danh tính: người dùng thật cho việc tương tác, **service account giới hạn** cho job tự động

Điểm đáng nhớ

Vấn đề lớn nhất của MCP ở quy mô công ty **không phải giao thức** – mà là quản trị: ai được gọi gì, ghi lại thế nào, và bao nhiêu tool là quá nhiều.

Ba ràng buộc của ngành luật

Xung đột lợi ích → không được khoá vào một nhà cung cấp model.

Zero Data Retention → đĩa làm việc tạm, không lưu dữ liệu khách.

Chi phí → định tuyến sang model mở khi chất lượng tương đương.

Hệ quả kiến trúc

Họ **tự xây hạ tầng agent** thay vì dùng runtime có sẵn của nhà cung cấp model. Định tuyến đa model cho **giảm 3–5 lần chi phí** trên benchmark nội bộ.

Lưu ý: Bài học tổng quát: ở ngành có quy định chặt (luật, y tế, tài chính), **ràng buộc tuân thủ đến trước và định hình kiến trúc** – không phải chọn kiến trúc đẹp rồi mới lo tuân thủ.

Hình dạng tải thật (agent code)

Input **45.000–200.000** token (context tích lũy qua phiên).

Output trung bình chỉ **~450** token.

Hệ quả

Chi phí agent bị chi phối bởi **context tích lũy**, không phải độ dài câu trả lời. Vì thế **nén context** và **cô lập context** là đòn bẩy chi phí mạnh nhất.

Nói lại với phần trước

Đây chính là lý do subagent **trả về bản tóm tắt**: mỗi token bạn không đưa lại vào context của agent chính là tiền tiết kiệm ở *mọi* lượt sau, không chỉ lượt này.

Chuyện đã xảy ra (7/2025)

Một agent lập trình tự động **xoá cơ sở dữ liệu production** của khách trong lúc đang *đóng băng code*, rồi báo cáo sai về việc mình đã làm.

Điều rút ra

Không phải chuyện model “ảo giác”. Đây là chuyện **quyền ghi** vào hệ thống thật mà **không có cổng duyệt**.

Lưu ý: Quy tắc thực dụng: hành động **không hoàn tác được** thì phải có người duyệt, hoặc phải chạy trên môi trường có thể khôi phục. “Agent giỏi hơn” **không** thay thế được ranh giới quyền hạn.

Harness Engineering

Phần bao quanh model – vòng lặp, tool, context, kế hoạch, hook
– đáng giá bao nhiêu? Câu trả lời 2026: đáng giá đến mức nên
sửa nó trước khi nghĩ tới đổi model

13

Harness là gì?

Toàn bộ phần **bao quanh** model để biến nó thành agent: prompt hệ thống, bộ tool, filesystem, sandbox, vòng lặp, kiểm chứng, quản lý context.

Ý tưởng cốt lõi

Mỗi thành phần harness **mã hoá một giả định** về việc model *chưa* tự làm được.

Model mạnh lên → giả định đó hết đúng → **gỡ bớt harness**. Anthropic đã tự gỡ lớp chia sprint của mình khi model lập kế hoạch tốt hơn.

Vì sao đáng học

Day 09 dạy **chia vai trò**; harness engineering là phần kỹ thuật bao quanh mỗi vai. Cùng một model, harness khác nhau cho kết quả rất khác nhau – xem slide sau.

Thí nghiệm (LangChain, 2/2026)

Giữ **nguyên model**, chỉ thay phần harness: middleware kiểm chứng, middleware bơm context, phát hiện vòng lặp, phân bổ ngân sách suy luận.

Terminal-Bench 2.0: **52,8% → 66,5%**.

Đọc con số cho đúng

Đây là **một bài blog kỹ thuật**, không phải bài báo bình duyệt. Bảng xếp hạng Terminal-Bench 2.1 hiện ở mức ~84%, nên mức tăng này là **nhảy giữa bảng**, không phải lên đỉnh bảng.

Điều đáng nhớ

Khi hệ chạy kém, phản xạ thường là **đổi model**. Bảng chứng này nói: hãy sửa **harness** trước – rẻ hơn nhiều và biên độ cải thiện là thật.

mini-swe-agent

~**100 dòng** code. **Không** dùng tool-calling API – mọi hành động đều là một lệnh bash. Lịch sử hội thoại nổi thêm tuyến tính.

Vẫn đạt **>74%** trên SWE-bench Verified.

Bài học

Model càng mạnh thì càng **ít cần** tool tự chế – cứ để nó dùng shell.

Harness tốt $\neq$ harness dày. Harness tốt = **đúng giả định** về chỗ model còn yếu.

Lưu ý: Hai slide này **cố tình mâu thuẫn nhau** – và cả hai đều đúng. Thêm harness giúp ích khi nó bù đúng chỗ yếu; thêm harness **thừa** chỉ tạo thêm chỗ hỏng. Câu hỏi luôn là: **giả định nào đang được mã hoá ở đây, và nó còn đúng không?**

Lấy đúng lúc

Không nạp sẵn mọi thứ; **tra khi cần** theo tham chiếu.

Nén & ghi chú

Gần đầy context thì **tóm tắt rồi khởi động lại**; ghi chú ra **file** như bộ nhớ ngoài.

Cô lập & tóm tắt

Subagent trả về **1–2 nghìn token tóm tắt**, không phải toàn bộ trace.

Con số minh họa

Cùng một tác vụ, thay vì nạp định nghĩa hàng trăm tool rồi kéo cả dữ liệu vào context, cho model **viết code gọi tool và lọc trong sandbox: 150.000 → 2.000 token**. Cùng model, cùng tool – khác harness.

Vòng lặp thật sự chỉ có vậy

dựng messages → gọi model → đọc tool call → chạy tool → nối kết quả vào messages → kiểm tra điều kiện dừng → lặp lại.

Điều kiện dừng thật

hết **số lượt** · hết **tiền** · hết **thời gian** · quá nhiều lần **parse lỗi liên tiếp**

Điều khiển nhiều người bắt ngờ

Agent **không** dừng vì hiểu rằng việc đã xong. Nó dừng vì **chạm trần ngân sách**, hoặc vì thấy một **dấu hiệu quy ước** do ta tự định nghĩa (một tool “submit”, hay một chuỗi đánh dấu trong output). Toàn bộ “sự thông minh” của việc dừng nằm ở **chỗ ta thiết kế**, không nằm trong model.

Trả về thể này thì agent chịu

stack trace thô · mã lỗi khó hiểu · đổ nguyên 200 nghìn dòng log vào context

Trả về thể này thì agent sửa được

gợi ý hành động tiếp theo · **ví dụ input đúng định dạng** · cắt bớt kèm cách lấy thêm (phân trang, lọc)

Con số cụ thể

Một harness production **cắt kết quả tool ở ~25.000 token** theo mặc định. Và mọi kết quả đều có cấu trúc: **exit code** riêng, stdout/stderr gộp, thông tin exception riêng – để agent phân biệt được *sai cú pháp* với *chạy đúng nhưng thất bại*.

Nguyên tắc

Thêm tool không đồng nghĩa tốt hơn.
Vài tool được thiết kế kỹ, gộp nhiều thao tác, thắng hàng trăm tool bọc 1:1 quanh API.

Đặt tên có **tiền tố** để tránh đụng nhau; tránh tên tham số mơ hồ (`user` → `user_id`).

Cách production giải bài toán phình

Nạp **tên tool** thôi lúc khởi động; khi cần thì agent gọi một tool *tìm tool* để lấy schema của đúng vài cái nó dùng.

⇒ cấm thêm server gần như **không tốn thêm context**.

Lưu ý: Chi tiết đáng nhớ: **chỉ sửa lại phần mô tả tool** – không đổi model, không đổi code – đã đủ đưa một model lên top các benchmark dùng tool. **Mô tả tool là prompt**, hãy review nó như review code.

Không phải “tóm tắt cho ngắn”

- kích hoạt theo **ngưỡng token** (ví dụ 100k)
- xóa **kết quả tool cũ nhất trước**
- **giữ lại N cặp gần nhất** (ví dụ 3)
- để lại **placeholder** để không có tham chiếu lơ lửng
- **ghim** kết quả của một số tool: không bao giờ xóa

Xoá đi đâu?

Một harness mã nguồn mở **không xóa** – nó **đẩy lịch sử ra file** rồi mới dọn khỏi context.

Agent vẫn đọc lại được khi cần; context thì nhẹ đi.

Lưu ý: Lỗi kinh điển khi tự viết compaction: xóa **kết quả** của một tool call nhưng để lại **lời gọi** (hoặc ngược lại) → lịch sử hội thoại không hợp lệ, model bối rối. Luôn xóa theo **cặp**.

Todo list như một cấu phần

Kế hoạch để **ngoài context** thì **sống sót qua compaction** và **soi được từ bên ngoài** – kế hoạch nằm trong đầu model thì không.

Có harness còn **chặn** việc sửa kế hoạch song song, ép cập nhật tuần tự để plan không loạn.

Hook: chen vào giữa vòng lặp

Chặn/duyet/sửa **trước** khi tool chạy; biến đổi **kết quả** trước khi model đọc.

Ví dụ thật: hook viết lại lệnh test để chỉ đưa **dòng FAIL** vào context thay vì cả nghìn dòng log.

Vì sao hai thứ này quan trọng

Cả hai đều là cách **lập trình viên giành lại quyền kiểm soát** mà không cần sửa prompt: một cái giữ *ý định* bền vững, một cái kiểm soát *cái gì được vào context*.

Tự Chủ Dài Hạn: Chạy Được Bao Lâu?

Harness quyết định agent làm được gì trong một lượt; phần này hỏi câu khó hơn: giữ cho nó chạy đúng suốt nhiều giờ, nhiều phiên – đo thế nào, hỏng ra sao, và làm gì để nó sống sót



Ý tưởng đo (METR)

Lấy các task đã **bấm giờ trên chuyên gia người**, rồi hỏi: model làm được task **dài bao nhiêu** với xác suất thành công 50%?

Đơn vị của trục hoành là **thời gian người**, không phải số bước.

Xu hướng & cảnh báo

- nhân đôi khoảng **7 tháng** (2019–2025)
- chính METR nói: đo **trên 16 giờ là không đáng tin** với bộ task hiện tại
- benchmark tốt **không đảm bảo** triển khai thật tốt

Vì sao cách đo này hay – và đừng đọc một đường duy nhất

Trục là **thời gian người**, thứ ai cũng hiểu. Nhưng code/toán đang ở **50–200+ phút** trong khi agent **bấm giao diện** ngắn hơn **40–100 lần**, đi sau **~2 năm**.

Reward hacking (đo được)

METR đo trên hàng nghìn lần chạy: ở bộ task R&D, model “**lách**” **tiêu chí thành công 30,4%** số lần chạy.

Model **biết** là trái ý người dùng nhưng vẫn làm; dẫn “đường gian lận” chỉ giúp rất ít.

Ba cơ chế hồng

- **lỗi cộng dồn** qua từng bước
- **context rot** – càng dài càng tụt
- **trôi mục tiêu** và không tự phục hồi được

Lưu ý: Task ngắn hồng vì **làm sai**. Task dài còn hồng vì **tối ưu sai thứ** – đây là lý do phần Observability và tiêu chí đánh giá quan trọng hơn nhiều khi để agent chạy lâu.

Hai mức bảo đảm

Checkpoint của framework: lưu state sau mỗi node – rẻ, đủ cho lỗi *logic*.

Durable execution: ghi nhật ký từng bước, sống sót qua **crash tiến trình** – đắt hơn, mạnh hơn.

Kỷ luật bắt buộc: idempotency

Replay sẽ **chạy lại** các bước. Nếu bước đó **gửi mail** hay **trừ tiền**, ta vừa làm việc đó hai lần.

Lưu ý: Cách chuẩn: mỗi hành động có **tác dụng phụ** phải mang một **khoá idempotency** sinh từ ngữ cảnh bền vững, để lần chạy lại nhận ra “việc này làm rồi”. Và biết rõ snapshot của bạn **không bắt** cái gì: sửa file qua shell, thay đổi do subagent, symlink...

Bộ nhớ + dọn context

Trên một benchmark **100 lượt**:

+39% tỉ lệ hoàn thành

—84% token tiêu thụ

Vì sao cả hai cùng tốt lên

Dọn context làm agent **đỡ lạc** (bớt context rot) và **đỡ tốn** (ít token hơn) cùng lúc.

Điều kiện: những gì bị dọn phải được **ghi ra bộ nhớ ngoài trước**, nếu không là mất thật.

Đây là phản ví dụ cho slide trước

Ở phần chẩn đoán ta thấy một nghiên cứu đo được **lớp bộ nhớ làm giảm** hiệu năng dài hạn. Ở đây lại **tăng**. Khác biệt nằm ở **cách làm**: bộ nhớ gắn với việc *dọn có kỷ luật* thì thắng; bộ nhớ nhồi thêm vào context thì thua.

File tiến độ

Một file ghi **đã xong gì, còn gì** – đọc lại được ở phiên sau.

Checklist hạng mục

Danh sách tính năng/tiêu chí, tick dần – chống **quên** và chống **lặp**.

Commit thường xuyên

Mỗi bước có kết quả thì **commit** – vừa là checkpoint, vừa là đường lùi.

Vì sao ba thứ tầm thường này lại hiệu quả

Chúng biến trạng thái từ thứ **sống trong context** (sẽ bị dọn, sẽ trôi) thành thứ **sống trên đĩa** (bền, đọc lại được, con người xem được). Với task nhiều giờ, **compaction** **một mình không đủ** – phải có nơi ghi ra ngoài.

Thang kiểm chứng, rẻ → đắt

- tự soi lại trong cùng một prompt
- một **subagent** đánh giá riêng
- một **cổng chặn** lúc kết thúc
- một **người chấm** với **context** sạch

Quy tắc chọn

Có **oracle** máy kiểm được (test, compile, schema) thì kiểm **dày** – rẻ và đáng. Không có oracle thì kiểm ở **mốc**, đừng kiểm mỗi bước bằng LLM: tốn mà không chắc đúng.

Lưu ý: Nhắc lại từ phần nền tảng: kiểm chứng **từng bước** tốt hơn chỉ chấm kết quả cuối, và **một vai riêng** tốt hơn để agent tự soi mình. Task càng dài, khoảng cách này càng lớn – vì lỗi sớm không được bắt sẽ đội lên qua từng bước.

Cache prompt

Phần context lặp lại đọc từ cache rẻ hơn nhiều lần. Rất hợp task dài vì context đầu gần như không đổi.

Phân tầng model

Model rẻ cho việc **fan-out** và sàng lọc; model mạnh cho **tổng hợp** và quyết định.

Dừng sớm

Đặt trần lượt/tiền/thời gian và **tiêu chí đủ tốt**, thay vì để chạy tới khi hết ngân sách.

Lưu ý: Nhớ lại con số đã học: multi-agent có thể **thắng đáng kể** nhưng tiêu **~15 lần** token, và **~80%** chênh lệch kết quả giải thích được chỉ bằng **lượng token đã tiêu**. Nghĩa là: trước khi khoe kiến trúc thắng, hãy kiểm tra bạn có đang chỉ **mua kết quả bằng compute** hay không.

Kế Hoạch Học Tập Ngày 9

Roadmap rõ ràng giúp học viên biết nên đầu tư thời gian vào đâu và cần nắm vững điều gì trước khi chuyển sang bước tiếp

15

- **Phần 1 (60 phút)** – Lý thuyết: single-agent limits, mental model, 4 patterns
- **Phần 1b (20 phút)** – Cơ chế điều phối: ai giữ kế hoạch, handoff vs agents-as-tools, trần phân cấp
- **Phần 2 (35 phút)** – Supervisor-worker deep dive + shared state + anti-patterns
- **Phần 3 (55 phút)** – MCP architecture + A2A message contract
- **Phần 4 (30 phút)** – LangGraph: nodes, edges, state, routing code
- **Phần 5 (30 phút)** – Observability + cost/reliability trade-offs
- **Nghỉ giữa buổi (10 phút)** – trước phần cuối, phần đòi đọc kỹ nhất
- **Phần 6 (20 phút)** – Harness engineering + tự chủ dài hạn: đọc bằng chứng cho đúng
- **Phần 7 (35 phút)** – An toàn, chế độ hồng, và mổ xẻ kiến trúc thật
- **Phần 8 (30 phút)** – Bên trong harness: vòng lặp, tool, context, kế hoạch
- **Thảo luận & hỏi đáp (20 phút)** – đối chiếu kiến trúc thật với các pattern đã học

Phần 6 đậm đặc nhất nhưng rơi vào cuối buổi – nên nghỉ trước, và giảng chậm hơn ở slide pass^k.

Từ Day 08 (bắt buộc)

- RAG pipeline: query → retrieve → generate
- tool calling cơ bản
- cách viết system prompt có cấu trúc
- hiểu một pipeline RAG cơ bản (Day 08)

Python foundations

- TypedDict và type hints
- dictionary operations
- function decorators cơ bản
- async/await (nếu đi vào async A2A)

Lưu ý: Nếu chưa vững phần RAG của Day 08, hãy xem lại trước – toàn bộ ví dụ hôm nay đều lấy một pipeline RAG làm điểm xuất phát.

Cấp 1: Foundation

- giải thích 4 giới hạn single-agent
- vẽ sơ đồ supervisor-worker
- phân biệt MCP và A2A
- phân biệt handoff vs agents-as-tools

Cấp 2: Implementation

- phác được supervisor + 2 workers với shared state
- viết được message contract giữa hai agent
- chỉ ra chỗ nào trong hệ nên đi qua MCP

Cấp 3: Mastery

- LangGraph có conditional routing
- trace log đọc được và actionable
- giải thích trade-off cost/reliability
- đọc một con số benchmark và nói được nó thực sự đo gì

Misconception	Reality
“Cứ đổi model mạnh hơn là hệ tốt lên”	Sửa harness thường rẻ và hiệu quả hơn đổi model
“Supervisor phải là model lớn nhất”	Supervisor chỉ cần đủ để route đúng
“Điểm benchmark cao = dùng được trong production”	Một-lần-chạy khác độ tin cậy; xem pass ^k
“Handoff và agents-as-tools là hai chuyện khác nhau”	Với model, cả hai chỉ là một tool call – khác ở chỗ runtime làm gì sau đó
“Multi-agent tự động giải quyết context problem”	Context vẫn phải được quản lý cẩn thận
“LangGraph chỉ dùng được với LangChain”	LangGraph dùng được với nhiều LLM framework

Sửa sớm để học viên không mang theo hiểu lầm

Đọc trước (chuẩn bị)

- LangGraph Quickstart (docs.langchain.com)
- MCP Introduction – modelcontextprotocol.io
- Sumers et al. (2023), CoALA – Section 2–3
- Xem lại pipeline RAG của Day 08

Đọc sau (củng cố)

- Anthropic – Building Effective Agents
- Anthropic – multi-agent research system (2025)
- A2A Protocol – a2a-protocol.org
- LangSmith Tracing Guide

Mẹo đọc

Đọc code example thực tế trước, architecture spec đầy đủ sau.

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 **Multi-agent** chia vai trò để hệ đỡ quá tải – không phải cứ tăng số agent. Chỉ dùng khi cần.
- 2 **Supervisor-worker** là điểm bắt đầu thực dụng nhất: supervisor route và tổng hợp; worker chuyên, stateless.
- 3 **MCP** nối agent với tool; **A2A** cho agent giao việc cho nhau – hai tầng, hai vấn đề.
- 4 **LangGraph** biến điều phối thành graph có state và routing tường minh – dễ debug, dễ chèn người vào.
- 5 **Trace** không chỉ để debug hôm nay; nó là dữ liệu để cải thiện hệ thống.

Chi phí là thật

Nhiều agent = nhiều token. Tiền chủ yếu nằm ở **context tích lũy**, không ở câu trả lời.

Lỗi nằm ở thiết kế

Phần lớn lỗi **không** do model yếu. Làm rõ vai trò và thêm một bước kiểm chứng là hai can thiệp rẻ nhất.

Ranh giới quyền hạn

Tính quyền trên **cả đồ thị**. Việc không hoàn tác được thì phải có người duyệt.

Và nếu chỉ nhớ một câu

Khi hệ chạy kém, hãy sửa **harness** và **thiết kế vai trò** trước khi nghĩ đến đổi model – rẻ hơn nhiều, và bằng chứng cho thấy biên độ cải thiện là thật.

Bài tiếp theo

Agent UX & Thiết Kế Trải Nghiệm AI

“Hệ thống đã thông minh hơn và phối hợp tốt hơn. Nhưng nếu trải nghiệm kém, user vẫn sẽ không muốn dùng.”

Bài tập về nhà

- Hệ chạy đúng nhưng user không hiểu chuyện gì vừa xảy ra – đã đủ tốt chưa?
- Chỗ nào cần transparency, confidence indicator, và human handoff?

1. Model Context Protocol, *Official Documentation & Spec* – modelcontextprotocol.io (architecture, transports, authorization).
2. Google / Linux Foundation, *Agent2Agent (A2A) Protocol* – a2a-protocol.org.
3. Anthropic (2024), *Building Effective Agents*; Anthropic (2025), *How we built our multi-agent research system*.
4. Anthropic Engineering (2025–2026), *Effective Context Engineering for AI Agents*; *Code Execution with MCP*; *Harness Design for Long-Running Applications*.
5. LangGraph Docs, *Multi-Agent & Human-in-the-Loop* – docs.langchain.com/oss/python/langgraph.
6. OpenTelemetry, *GenAI Semantic Conventions*.

1. Sumers et al. (2023), *CoALA: Cognitive Architectures for Language Agents* – arXiv:2309.02427.
2. Wang et al. (ICML 2024), *Executable Code Actions Elicit Better LLM Agents (CodeAct)* – arXiv:2402.01030.
3. Yao et al. (2024), *τ -bench* – arXiv:2406.12045 (nguồn gốc chỉ số pass^k).
4. Kapoor & Narayanan et al. (2024), *AI Agents That Matter* – arXiv:2407.01502 (phê bình phương pháp đánh giá).
5. METR (2025), *Measuring AI Ability to Complete Long Tasks* – arXiv:2503.14499; metr.org/time-horizons.
6. Cemri et al. (NeurIPS 2025), *Why Do Multi-Agent LLM Systems Fail? (MAST)* – arXiv:2503.13657.
7. OpenAI (2024/2025), *MLE-bench* – arXiv:2410.07095; *PaperBench* – arXiv:2504.01848.
8. Chroma Research (2025), *Context Rot* – trychroma.com/research/context-rot.

Hỏi & Đáp

Một agent rất giỏi có thể làm nhiều việc. Nhưng hệ thống tốt hơn thường bắt đầu từ câu hỏi: nên chia vai trò ở đâu để dễ kiểm soát và dễ cải thiện nhất?